L01: Introduction to Neural Network with Logistic Regression

Tan Hung Khoon
FICT, UTAR

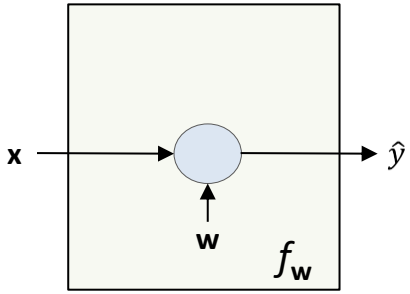
Table of Contents

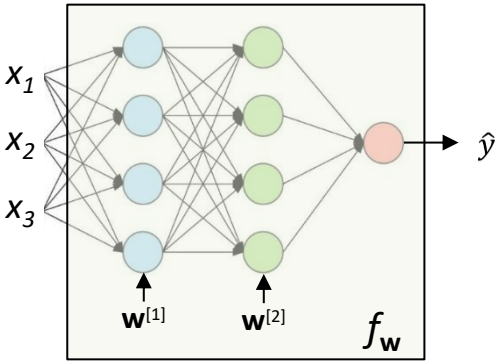
1. Introduction to Neural Network
2. Logistic Regression: A Neural Network Perspective
3. Cost Function
4. Gradient Descent

Introduction to Neural Network

What is a neural network?

- Deep learning refers to (very large) neural network.
- Neural network is a structure that consist of one or more layers of one or more computational units called neurons.

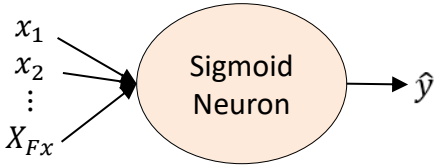


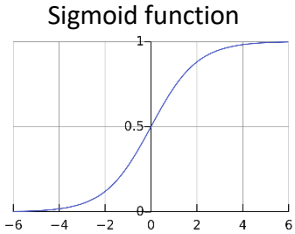


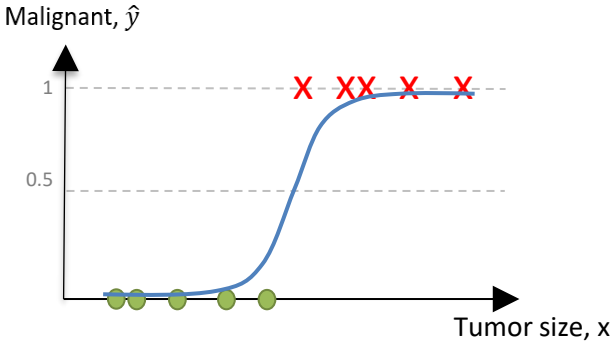
- Neural network can be viewed as a function $\hat{y} = f_{\mathbf{w}}(\mathbf{x})$ that predicts the output $\hat{y}$ given some input **x**.
- The function is parameterized by the parameters **w** which controls how the function works.

Neuron

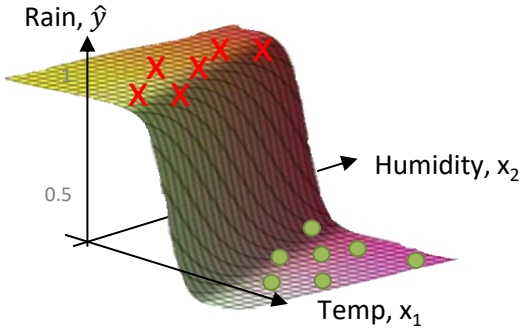
- A neuron is the basic computational unit in a neural network.
- It fits a particular function to model different kinds of data. For example, a sigmoid neuron can fit a sigmoid-like function for different tasks.







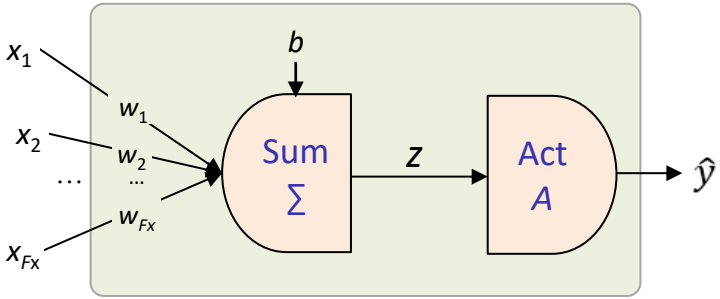
Tumor classification



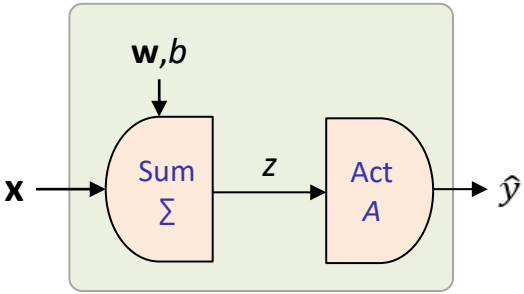
Rain prediction

Operations of a neuron

- Given an input with F_x features, $\mathbf{x} \in \mathbb{R}^{F_x \times 1}$ a neuron computes the predicted output, $\hat{y} \in \mathbb{R}$ in two stages:



Non-vectorized view

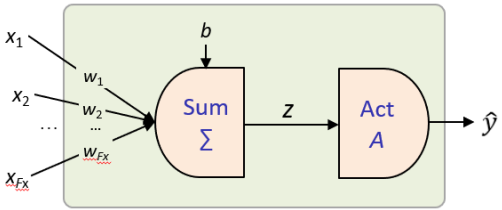


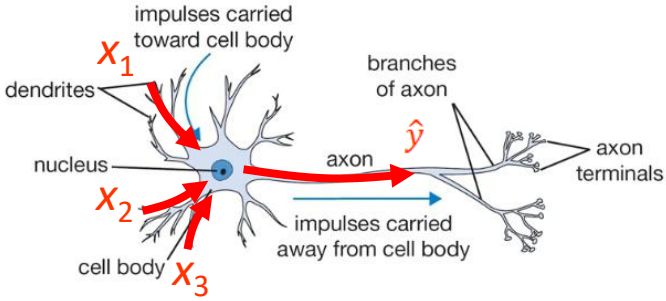
Vectorized view

- Summation phase combines the input features to generate the weighted input $z \in \mathbb{R}$. The network parameters are the weights $\mathbf{w} = [w_1, \dots, w_{F_x}] \in \mathbb{R}^{F_x \times 1}$ and bias b.
- Activation phase transforms z non-linearly into the output signal $\hat{y}$ with an activation function. Non-linearity transformation allows the neural network to model complex functions

Analogy to the brain

- Neural network is inspired by goal of modeling biological neural systems.

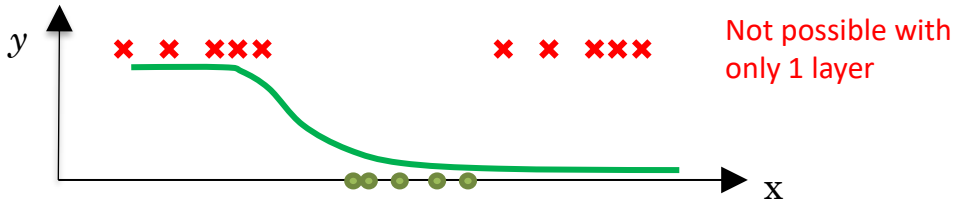




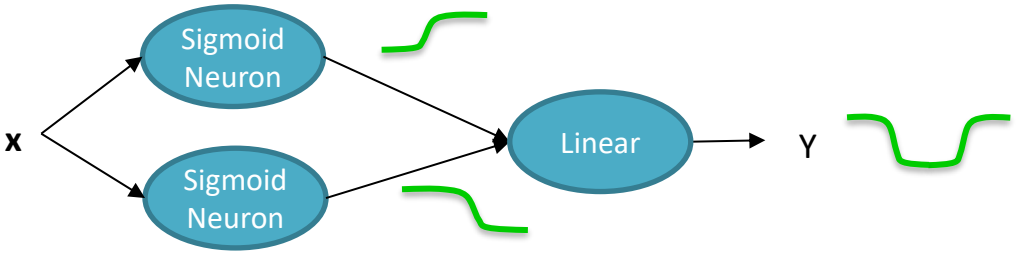
- The analogy to the brain has been overstretched.
  - Not a computational model of the brain.
  - Neural network is a flexible function $\mathbf{x} \rightarrow \hat{y}$ that can map different kinds of task.

Why stack neurons?

- Neural network with only one layer cannot model complex functions.



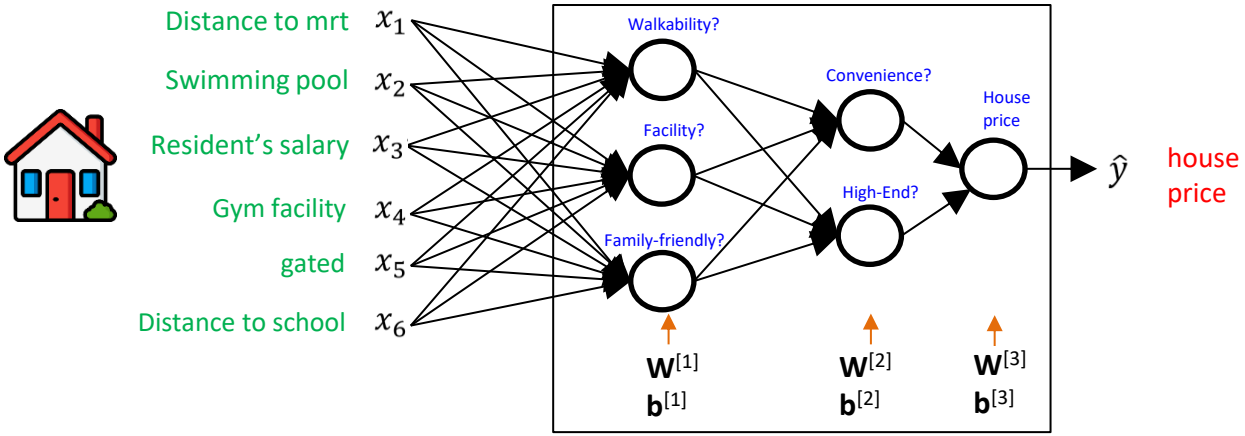
- Universal Approximation Theorem: A neural network with two layers can theoretically approximate any continuous function.



- But deeper network allows us to model complex function more efficiently (same performance with less neurons and parameters).

Neural Network (NN)

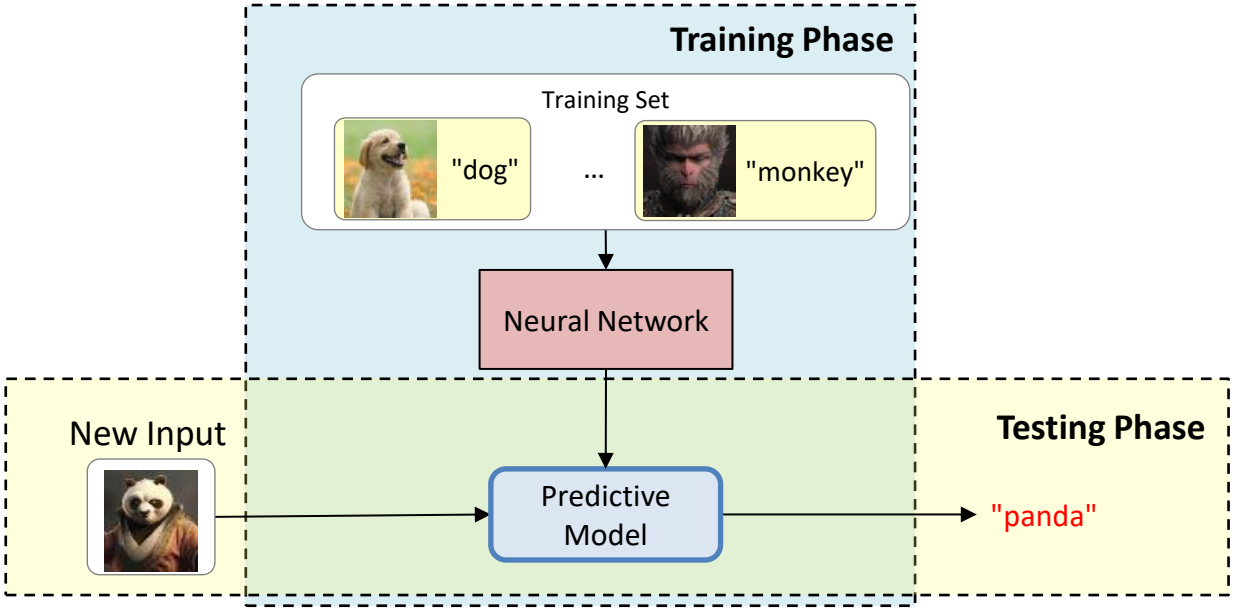
- Neural network is constructed by stacking layers of neuron.



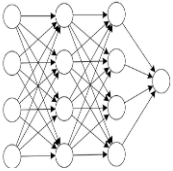
- Input layer receives the input features **x** of a sample.
- Hidden layer neurons (first two layers) extracts useful intermediate features.
- Output layer (last layer) predict the output $\hat{y}$ given intermediate features.
- The network is parameterized by the network parameters $\mathbf{W} = \{\mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]}\}$ and $\mathbf{b} = \{\mathbf{b}^{[1]}, \dots, \mathbf{b}^{[L]}\}$
- The correct model (specific values of **W** and **b**) is acquired through training. To train the model, we need sufficient training samples.

Supervised Learning with Neural Network

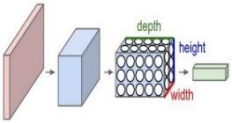
- Neural network is typically trained using supervised learning.
- Supervised learning - the training set comes with input labels.
- Commences in two phases:
  - Training phase - trains a model on labeled data
  - Testing phase – uses the model to predict on new input data



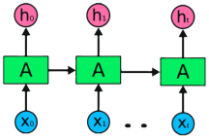
Types of neural Network



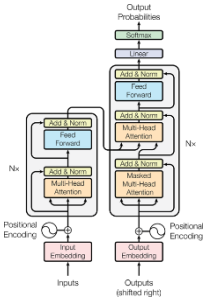
Standard Neural Network



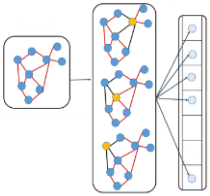
Convolutional Neural Network (CNN)



Recurrent Neural Network (RNN)



Transformer



Graph Convolutional Network (GCN)

- Different NN are good at handling different types of data, especially on unstructured data compared to other ML algorithms

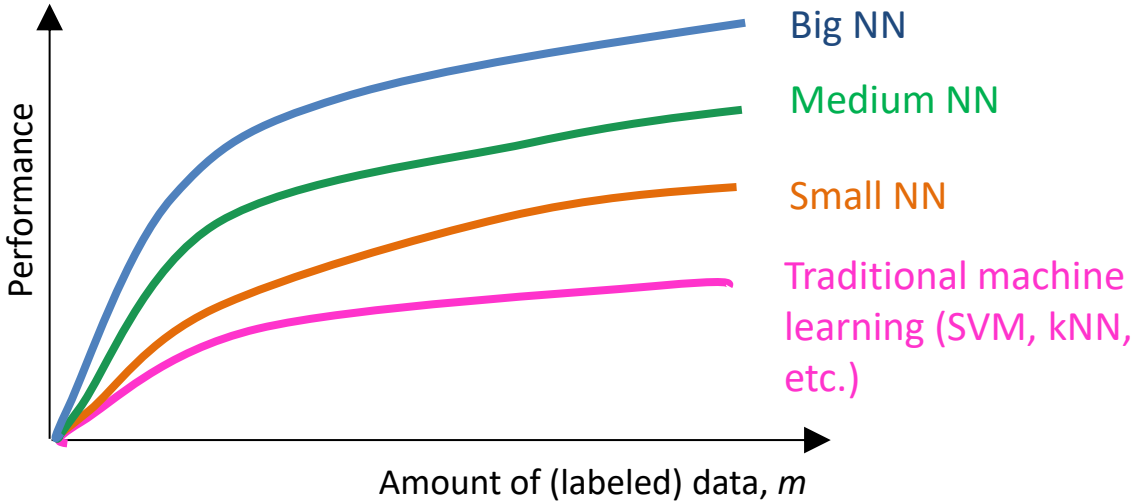
Data Type			Neural Network Type
Structured	Attribute	Table	Standard Neural Network (NN)
	Relational	Social Network	Graph Convolutional Network (GCN)
Unstructured	Spatial	Image, video	Convolutional Neural Network (CNN) Visual Transformer (ViT)
	Sequence	Text, audio, Video	Recurrent Neural Network (RNN) Long-short Term Memory (LSTM) Transformer

Why is deep learning taking off?

- Big data
  - More data (Smart phones, digital economy, IoT)
  - Neural network scales very well with data (next slide)
- Faster computation
  - GPU, TPU, AI accelerators (more than 50x speedup compared to CPU)
- Better algorithms
  - Better architectures (transformer, etc.)
  - Better backpropagation (BatchNorm, skip connection, etc.)
  - Better optimizers (momentum, Adam, AdamR, etc.)
  - Better initialization (Xavier, Kaiming, etc.)
  - Better regularization (dropout, blockout, etc.)

Why is deep learning taking off?

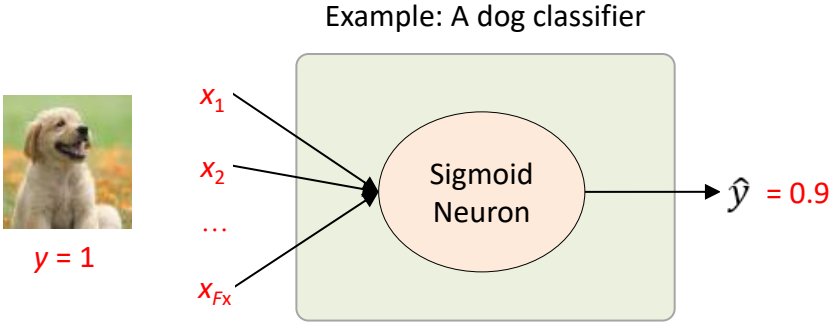
- Scale (big data, bigger network) is driving progress in DL
- Neural Network scales very well with big data compared to traditional machine learning algorithms



Logistic Regression

A Neural Network Perspective

Logistic Regression



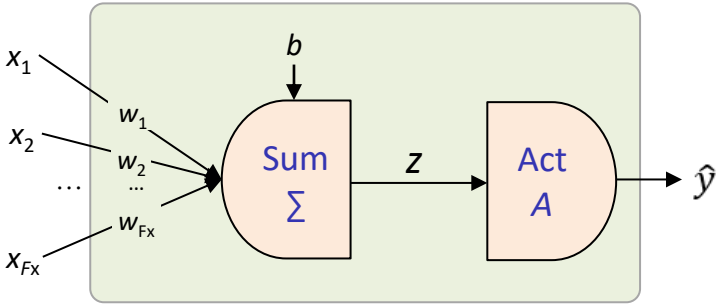
- Logistic Regression is a one-layered neural network with one neuron with sigmoid activation.
- It is a binary classifier. The label (or ground truth) y is a binary value:

$$y = \begin{cases} 1 & \text{if positive sample} \\ 0 & \text{if negative sample} \end{cases}$$

- The input $\mathbf{x} \in \mathbb{R}^{F_x \times 1}$ is a column vector with F_x features
- The targeted variable or output score $\hat{y} \in \mathbb{R}$ is a scalar (real value).

Inference (1 sample)

- Inference is the process of computing the output scores $\hat{y}$ given an input feature $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ X_{Fx} \end{bmatrix}$
- Logistic regression performs inference in two stages:



  - Summation phase: Linearly combines the input features in **x** to get the weighted input *z*. The inputs are weighted by the parameters **w**.
  - Activation phase: Transforms the weighted input *z* with the activation function to get the predicted output $\hat{y}$

Summation Phase (1 sample)

- Given one sample $\mathbf{x} \in \mathbb{R}^{F_x \times 1}$ (a column vector), the summation phase combines x_i linearly to get the weighted input $z \in \mathbb{R}$ (a scalar).

$$z = w_1 x_1 + \cdots + w_{F_x} x_{F_x} + b$$

where w_i is the weight for x_i and b is the bias.

- It can be implemented efficiently in the equivalent vectorized form:

$$z = \mathbf{x}^T \mathbf{w} + b$$

where

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_{F_x} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_{F_x} \end{bmatrix}$$

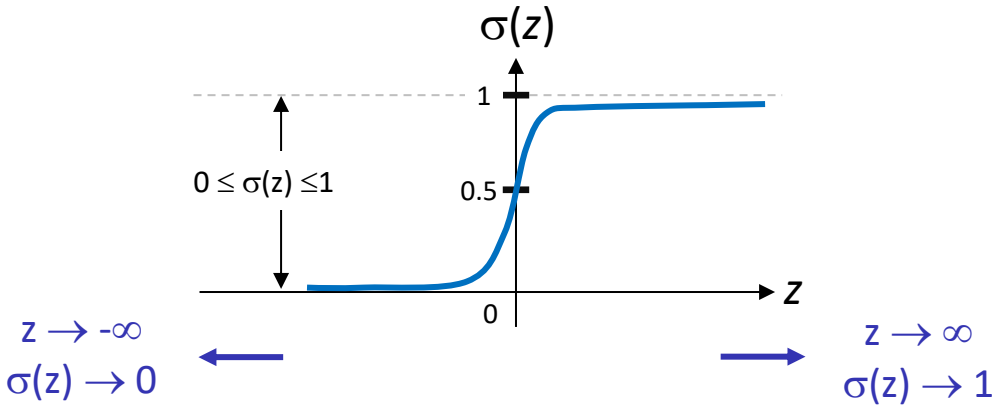
- **w** and b are the parameters of the model. Different values of **w** and b represent different models.

Activation Phase (1 sample)

- The activation phase performs non-linear transformation of the weighted input z with a selected activation function, e.g., sigmoid, ReLU, GeLU, maxout, etc.)
- Logistic Regression uses the sigmoid function as activation:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where z and $\hat{y}$ are scalars.



Input matrix and Label vector (*M* samples)

- Given *M* samples $\{(\mathbf{x}^{(1)}, y^{(1)}), \ldots (\mathbf{x}^{(M)}, y^{(M)})\}$, stack the transposed feature $\mathbf{x}^{(i)T}$ and $y^{(i)}$ vertically to get the input matrix **X** and label vector **y**.

$$\mathbf{X} = \begin{bmatrix} — & \mathbf{x}^{(1)T} & — \\ — & \mathbf{x}^{(2)T} & — \\ & \vdots & \\ — & \mathbf{x}^{(M)T} & — \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(M)} \end{bmatrix}$$

where

- $\mathbf{x}^{(i)}$ is the input feature for the *i*-th sample
- $y^{(i)}$ is the label for the *i*-th sample
- $\mathbf{X} \in \mathbb{R}^{M \times F_x}$ is the input matrix
- $\mathbf{y} \in \mathbb{R}^{M \times 1}$ is the label vector

- The *i*-th row of the input matrix, **X**[*i*, :], is the feature vector of the *i*-th sample while **y**[*i*] is its corresponding label.

Summation Phase (*M* samples)

- The vectorized version of the summation phase allows us to compute the weighted inputs $\mathbf{z} \in \mathbb{R}^{M \times 1}$ for all samples in parallel.

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b$$

$$\begin{bmatrix} z^{(1)} \\ z^{(2)} \\ \vdots \\ z^{(M)} \end{bmatrix} = \begin{bmatrix} — & \mathbf{x}^{(1)T} & — \\ — & \mathbf{x}^{(2)T} & — \\ & \vdots & \\ — & \mathbf{x}^{(M)T} & — \end{bmatrix} \begin{bmatrix} | \\ \mathbf{w} \\ | \end{bmatrix} + b$$

$$= \begin{bmatrix} \mathbf{x}^{(1)T}\mathbf{w} + b \\ \mathbf{x}^{(2)T}\mathbf{w} + b \\ \vdots \\ \mathbf{x}^{(M)T}\mathbf{w} + b \end{bmatrix}$$

- $z^{(i)}$ is the weighted input (scalar) for the *i*-th sample.
- $\mathbf{z}$ is the column vector storing the weighted input for all samples

Activation Phase (*M* samples)

- For batch data, the vectorized version performs element-wise operations for all samples in parallel.

$$\hat{\boldsymbol{y}} = \begin{bmatrix} \hat{y}^{(1)} \\ \vdots \\ \hat{y}^{(M)} \end{bmatrix} = \begin{bmatrix} \sigma\left(z^{(1)}\right) \\ \vdots \\ \sigma\left(z^{(M)}\right) \end{bmatrix}$$

$$= \sigma(\mathbf{z}) = \frac{1}{1 + e^{-\mathbf{z}}} \quad \text{(element-wise operation)}$$

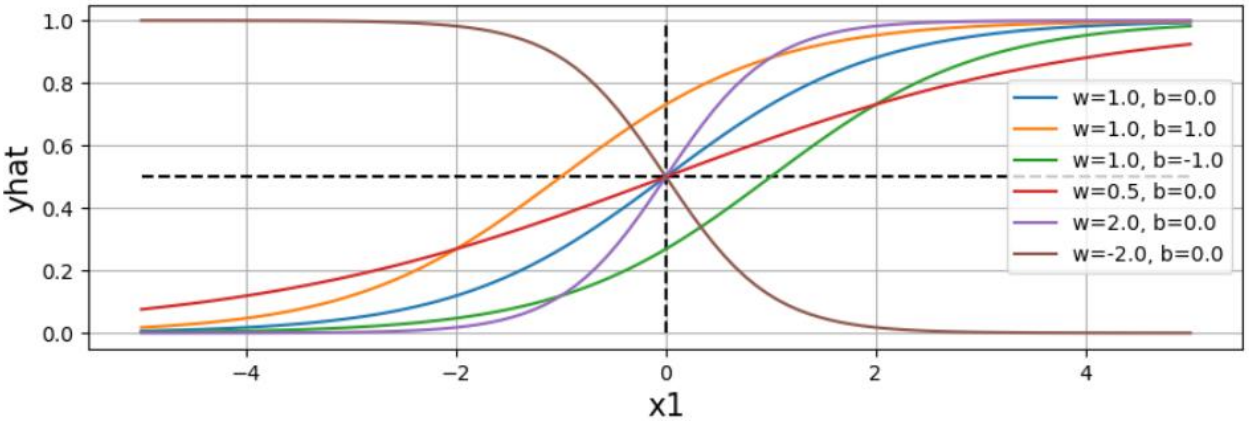
where $\mathbf{z} \in \mathbb{R}^{M \times 1}$ and $\hat{\mathbf{y}} \in \mathbb{R}^{M \times 1}$

Modeling with Logistic Regression

- Combining the summation and inference phase, the complete inference formulation are given by:

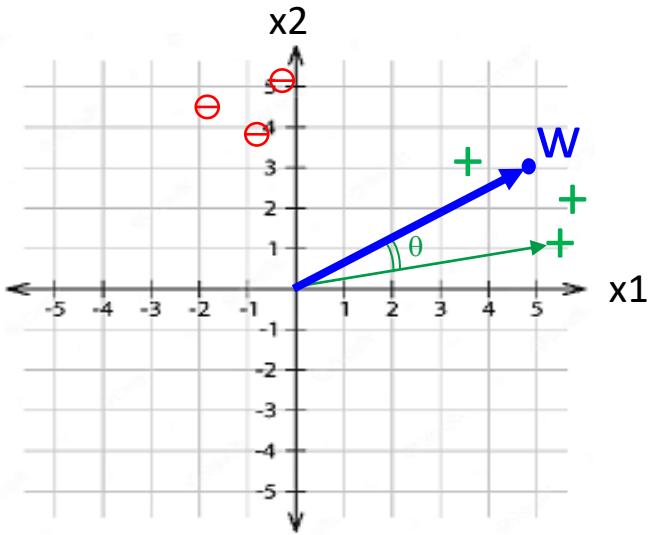
$$\hat{\mathbf{y}} = \frac{1}{1 + e^{-(\mathbf{X}\mathbf{w}+b)}}$$

- We can get different models by tweaking the parameter values **w** and *b*.



Interpreting Logistic Regression

- During training, the weights **w** of Logistic Regression is like a template vector to represent the positive samples (+).



- Each sample represents a point in the feature space.
- The weight represents the template for positive samples
- The dot product gives us the angular similarity between the samples and the weights
- Logistic Regression finds the **w** with high similarity with most positive samples, and vice versa

Example

Given the input matrix $\mathbf{X} = \begin{bmatrix} -5 & 3 \\ 0 & -2 \\ 5 & 4 \end{bmatrix}$ and the parameters for Logistic Regression are $\mathbf{w} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$, b = 1.

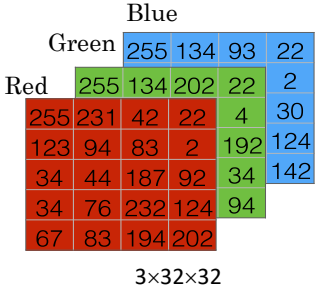
1. How many samples are in the data set? How many features do each sample have?

2. Compute the weighted input *z*.

3. Compute the predicted output of Logistic Regression.

Example

Given an image dataset of RGB images with resolution 32×32. Specify how to preprocess the images to build a cat classifier with Logistic Regression.

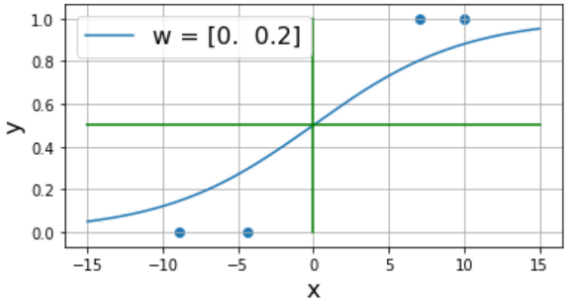


Cost Function

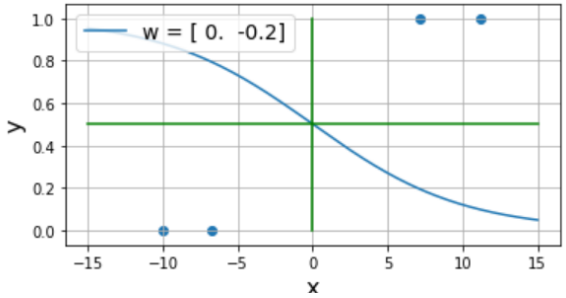
What is Cost Function?

- The cost function $J(\mathbf{w}, b)$ measures how well the model $(\mathbf{w}, b)$ fits the training set
  - J is a scalar where lower value is desirable.
  - J is low when the model fits the training set well, i.e., $\hat{y}^{(i)} \approx y^{(i)}, \forall i$

- Example:



J should be low



J should be high

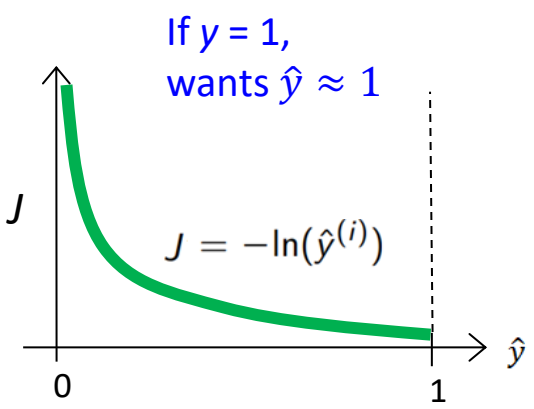
Binary Cost Entropy (BCE)

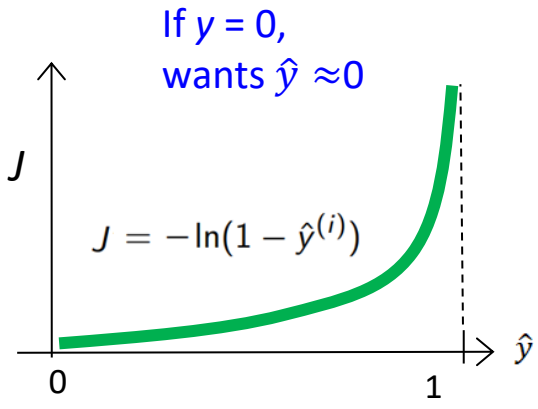
- For binary classification, use Binary Cost Entropy (BCE) as the cost function:

$$J(\mathbf{w}, b) = \frac{1}{M}\sum_{i=1}^{M} -y^{(i)} \ln\left(\hat{y}^{(i)}\right) - \left(1 - y^{(i)}\right) \ln\left(1 - \hat{y}^{(i)}\right)$$

$$= \text{mean}(-\mathbf{y} * \ln(\hat{\mathbf{y}}) - (1 - \mathbf{y}) * \ln(1 - \hat{\mathbf{y}}\ ))$$

- Cost for sample *i*:





Example

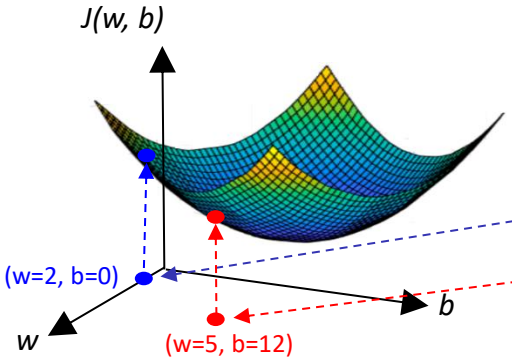
Given four samples, Logistic Regression predicts $\hat{y} = \begin{bmatrix} 0.11 \\ 0.15 \\ 0.93 \\ 0.95 \end{bmatrix}$. The ground truth label is $y = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$. Compute the binary cross entropy.

Parameter space vs Feature space

- Optimization (training) commences in the parameter space.
- Inference commences in the feature space.

Parameter space (Training)

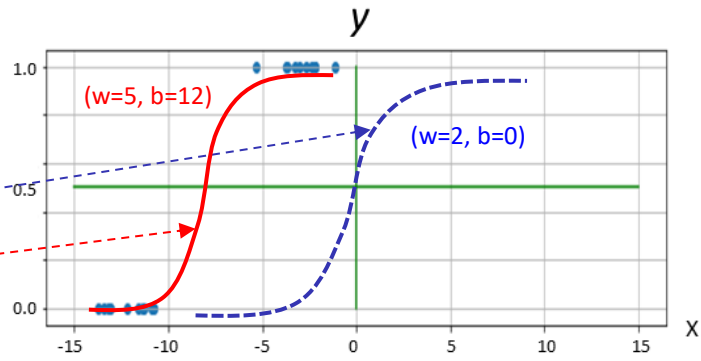
$$J(\mathbf{w}, b) = \frac{1}{M}\sum_{i=1}^{M} -y^{(i)} \ln\left(\hat{y}^{(i)}\right) - \left(1 - y^{(i)}\right) \ln\left(1 - \hat{y}^{(i)}\right)$$



Dependent variable: J
Independent variables: $\mathbf{w}$, b
Constant: $\mathbf{X}$, $\mathbf{y}$

Feature space (Inference)

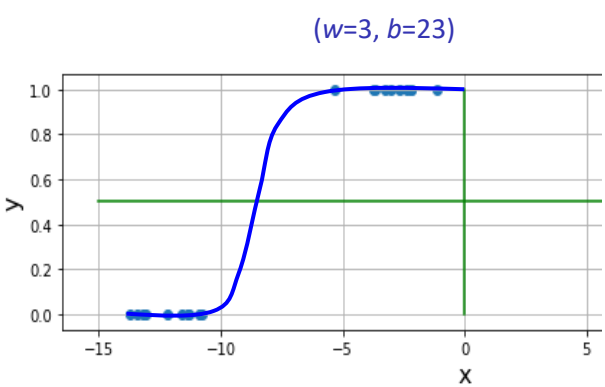
$$\hat{y} = \frac{1}{1 + e^{-(\mathbf{X}\mathbf{w}+b)}}$$

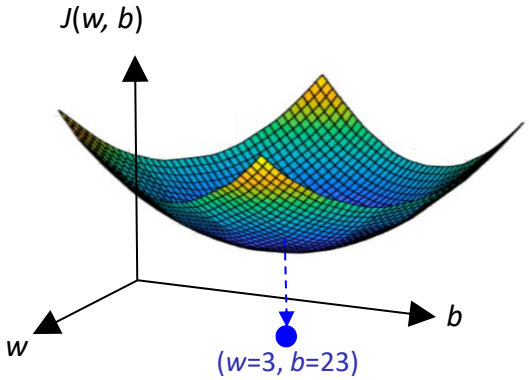


Dependent variable: $\hat{y}$
Independent variables: $\mathbf{X}$
Constant: $\mathbf{w}$, b

Training the model

- During training, the aim is to find the model (**w***, *b**) that minimizes *J*(**w**, *b*).
  - The minimum point of the cost surface in the parameter space
  - The curve that best fit the data in the feature space



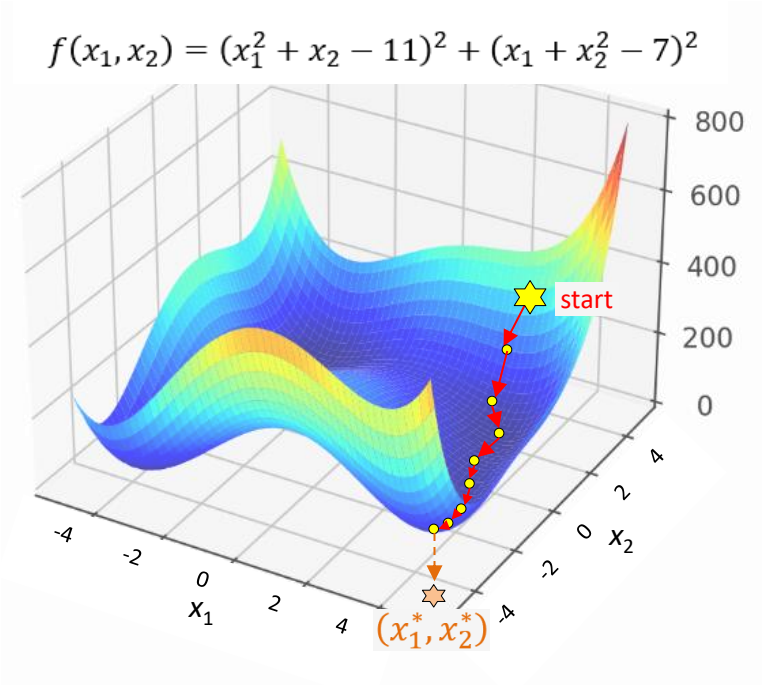


- How? Use an optimization algorithm (optimizer) to find the min point.
- The de facto optimizer for NN is a variant of Gradient Descent.

Gradient Descent

Gradient Descent

- Gradient descent is an optimization algorithm to find a local minimum $(x_1^*, \dots, x_n^*)$ of a differentiable function $f(x_1, \dots, x_n)$.

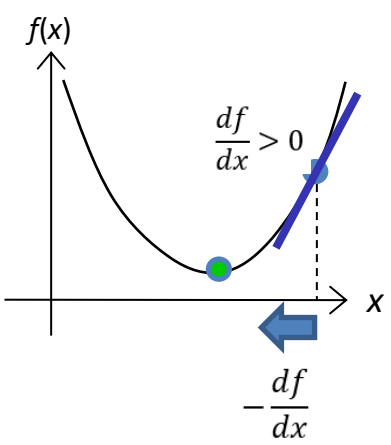


Strategy:

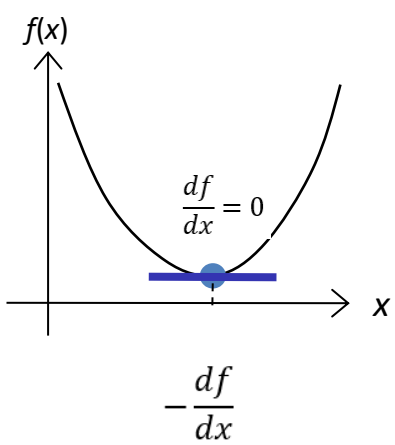
1. Initialize (x_1, …, x_n) based on some initialization scheme, e.g., randomly.
2. Make one step along the *steepest descent* direction to get a lower f.
3. Repeat step 2 until the minimum point $(x_1^*, \dots, x_n^*)$ is reached.

Direction of the steepest direction

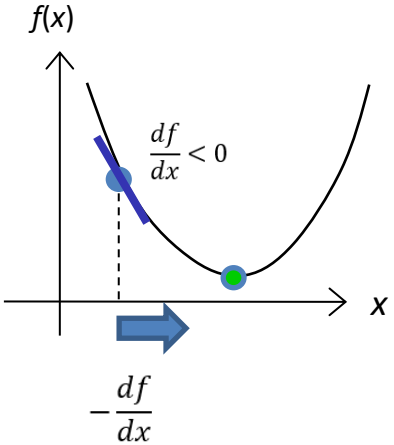
- Direction of the steepest descent is given by the negative gradient of f with respect to x, or $-\frac{df}{dx}$.



Should move to left



Should stay



Should move to right

- In general, update w following the direction $-\frac{df}{dx}$

Gradient Descent (for any differentiable functions)

Algorithm Gradient Descent (for any differentiable functions)

1: **function** GRADIENTDESCENT(α, iterations)
2: Initialize $\mathbf{x} \equiv \{x_1, \cdots, x_{N_x}\}$
3: **for** $t = 1$ to num_iterations **do**
4: $\frac{\partial f}{\partial x_1}, \cdots, \frac{\partial f}{\partial x_{N_x}} \leftarrow$ compute_grad($\mathbf{x}$)
5: $\forall i : x_i \leftarrow x_i - \alpha \frac{\partial f}{\partial x_i}$
6: **end for**
7: **return x**
8: **end function**

- $\frac{\partial f}{\partial x_i}$ is the gradient of the output f w.r.t. the input x_i
  - $-\frac{\partial f}{\partial x_i}$ is the direction of steepest descent.
  - Moving in the direction of the steepest descent reduces the value of $f(\mathbf{x})$.
- α is the learning rate
  - Controls how aggressive to update the parameters. Higher learning rate means more aggressive update.

Example

Use gradient descent to find the value (x_1^*, x_2^*) that minimizes

$$f(x_1, x_2) = x_1^2 + x_2^2 - x_1 x_2$$

Show your steps up to 3 iterations only. Initialize $(x_1, x_2) = (2, 5)$ and set the learning rate $\alpha = 0.1$. Round your answer to 2 decimal places in each step.

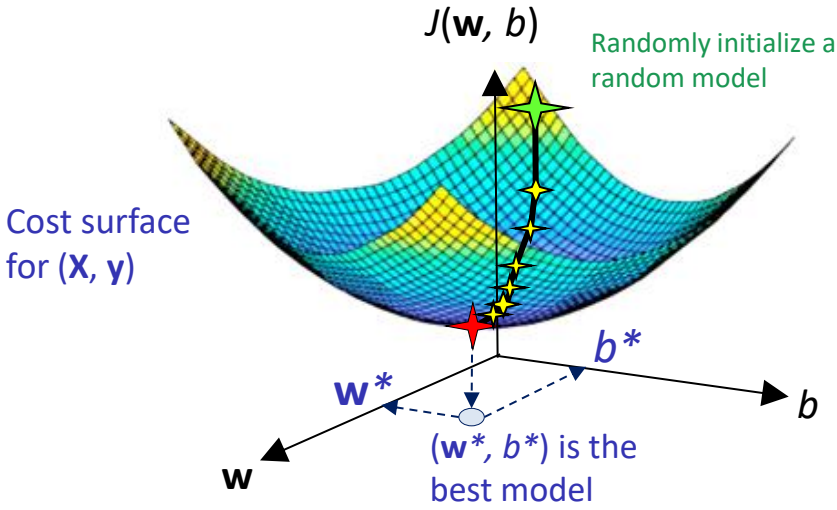
Solution:

$$\frac{df}{dx_1} = \qquad \frac{df}{dx_2} =$$

t	(x_1, x_2)	$f(x_1, x_2)$	$\left(\frac{df}{dx_1}, \frac{df}{dx_2}\right)$
1	(2, 5)		
2			
3			

Batch Gradient Descent (BGD) for Logistic Regression

- Given a training set (**X**, **y**), gradient descent finds the parameter values (**w***, *b**) that minimizes the cost $J(\mathbf{w}, b)$.
- Optimization happens in the parameter space



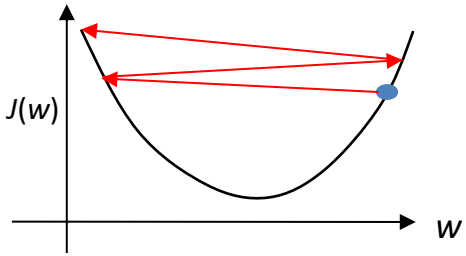
The Batch Gradient Descent (BGD) Algorithm

Algorithm Batch Gradient Descent (BGD) for training LogReg

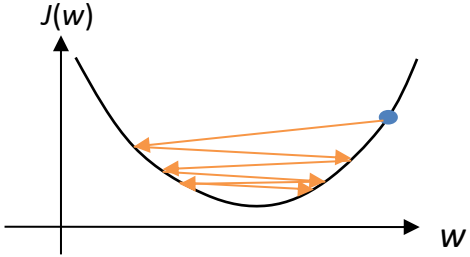
1: **function** BATCHGRADIENTDESCENT($\mathbf{X}, \mathbf{y}, \alpha,$ iterations)
2: Initialize $\mathbf{w}, b$
3: **for** $t = 1$ to num_iterations **do**
4: $\hat{\mathbf{y}} \leftarrow$ forward$(\mathbf{X}, \mathbf{w}, b)$
5: $J(\mathbf{w}, b) \leftarrow$ compute_cost$(\hat{\mathbf{y}}, \mathbf{y})$
6: $\frac{\partial J}{\partial \mathbf{w}}, \frac{\partial J}{\partial b} \leftarrow$ backward$(\mathbf{X}, \mathbf{y}, \hat{\mathbf{y}})$
7: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}$
8: $b \leftarrow b - \alpha \frac{\partial J}{\partial b}$
9: **end for**
10: **return** x
11: **end function**

- Steps 4 to 5: Perform forward propagation to predict on all training samples and compute cost
- Step 6: Compute the gradient of parameters with the backpropagation algorithm
- Steps 7 to 8: Update the network parameters
- Step 3: Repeat steps 4 to 8 until convergence

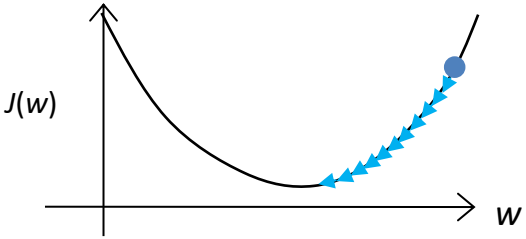
Setting the learning rate



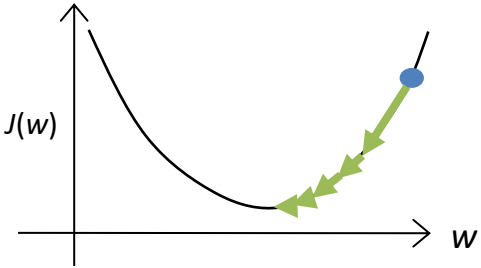
α is too big: J(w) increases over iteration until overflow happens



α is big: J(w) decreases over iterations but unable to reach the local minima point



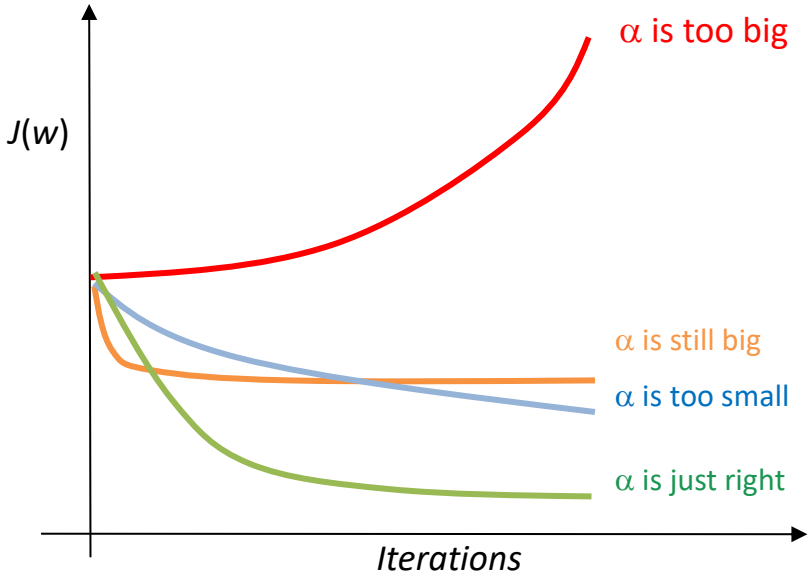
α is too small: J(w) decreases over iteration and finds the local minima point slowly.



α is just right: J(w) decrease over iteration and finds the local minima point efficiently.

Monitoring training progress

- Not possible to plot the cost surface in high-dimensional space.
- To monitor the learning progress, plot the cost-iteration plot.

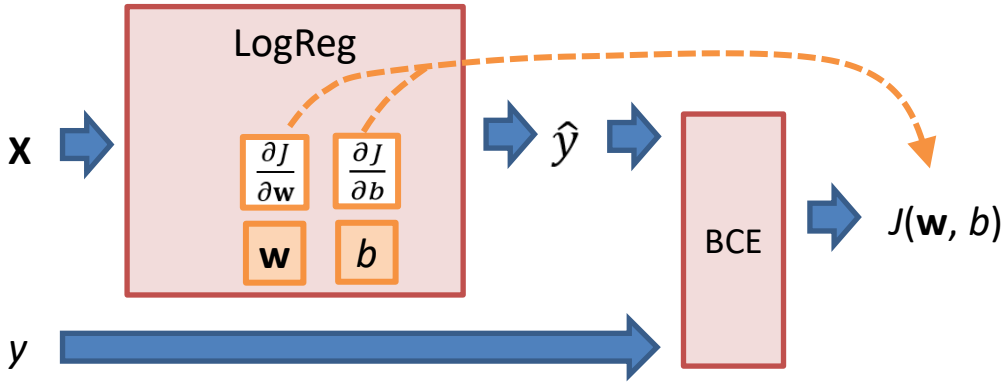


Stop immediately and re-run experiment with lower α

All three plots looks similar when plot individually. How do I choose?
Empirically through comparison with other runs or by experience from previous runs.

What does the gradient of **w** and b tell us?

- The polarity of the gradient of **w** and *b* tells us if **w** and *b* should be increased or decreased to minimize the cost *J*.



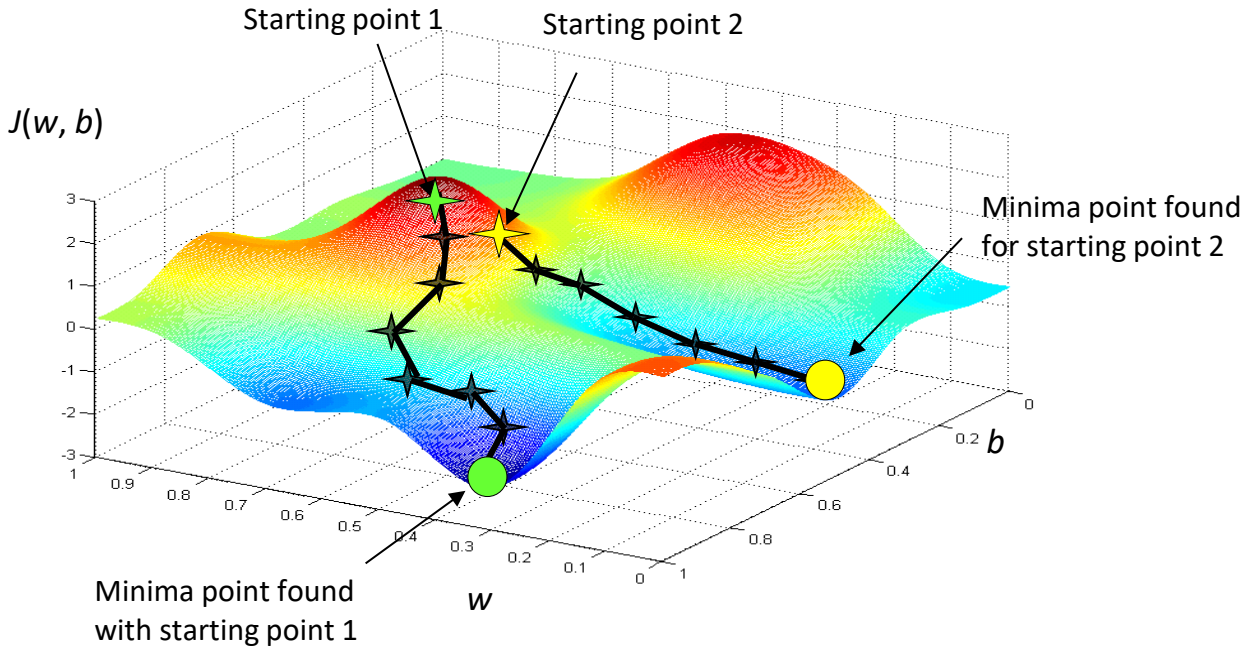
For example, if $\frac{\partial J}{\partial \mathbf{w}}$ is +ve, decreasing **w** will decrease *J*.

but if $\frac{\partial J}{\partial \mathbf{w}}$ is -ve, increasing **w** will decrease *J*.

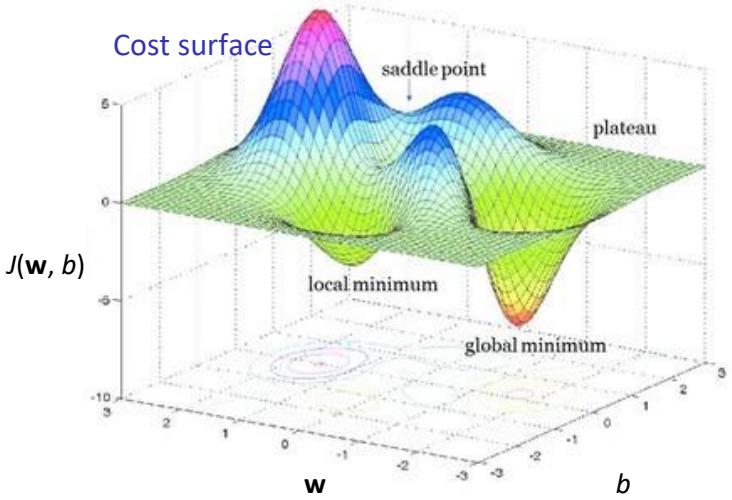
In both cases, moving **w** in the direction of - $\frac{\partial J}{\partial \mathbf{w}}$ decreases *J*.

Sensitivity to Initialization Point

- Gradient descent is sensitive to initialization. Performance varies depending on initial point.
- Important to find a good one. Different initialization schemes to be covered later.



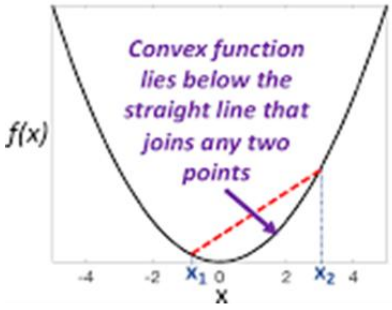
Global minima, Local minima, Saddle Point and Plateau



- Local minima – a point where the function value that is smaller than nearby points, but possibly greater than distant point.
- Global Minima – a point where the function value is smaller than all other feasible points.
- Saddle point - a point which is a local maxima in one direction but local minima in another.
- Plateau – large region of cost surface where the variance of the gradient is almost 0.

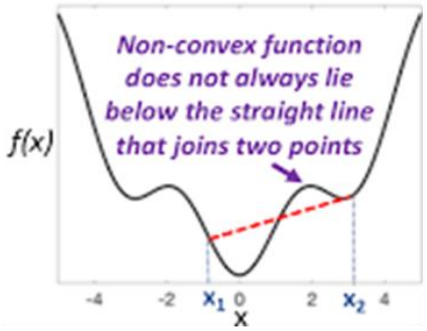
Local Minima and Global Minima

Convex function



Gradient descent finds the global minima if the cost function is convex (Logistic Regression)

Non-convex function

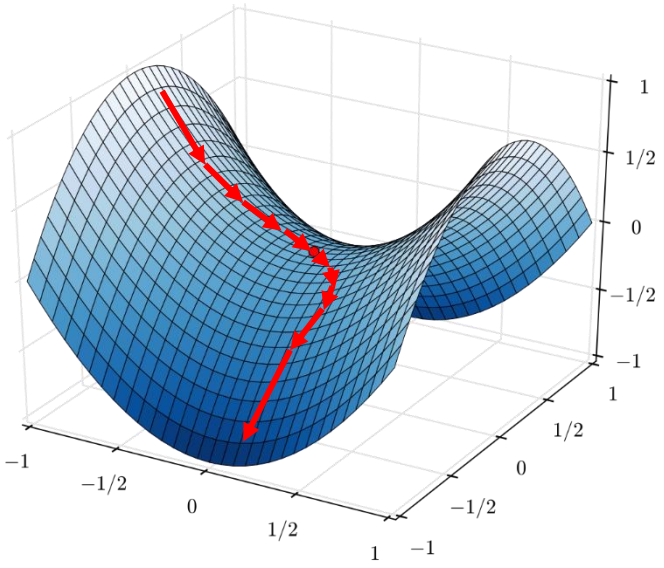


Gradient descent finds local minima for non-convex cost function (Almost all neural networks).

- Local minima is not an issue in deep learning:
  - Not common in high-dimensional space - rare for all dimensions to have minimum at the same point.
  - Local minima may be more desirable due to less overfitting.
  - A lot of good local minimum.

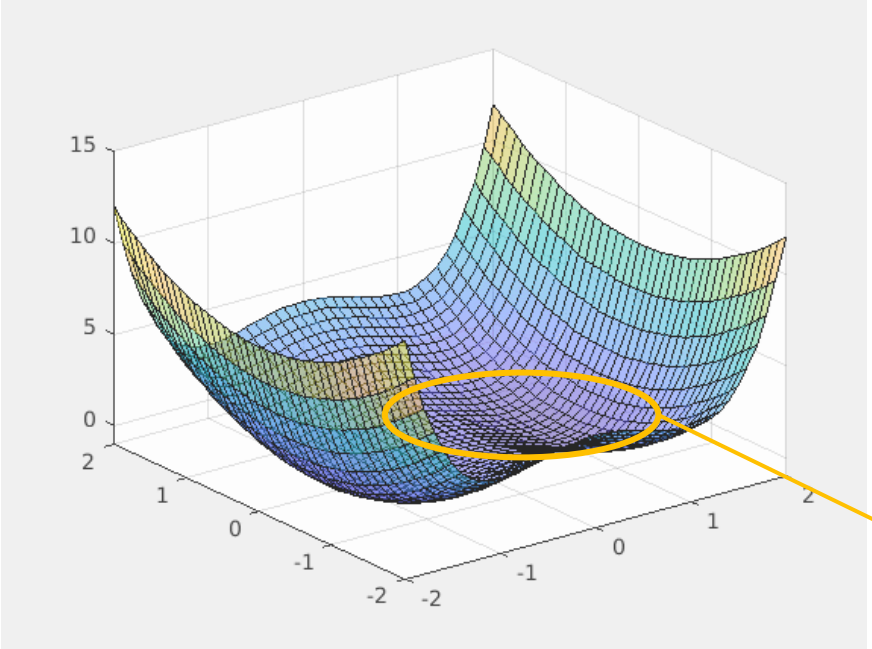
Saddle point

- Saddle point is a point which is a local maxima in one direction but local minima in another.
- More common in high-dimensional than local minima.
- Not an issue as saddle points are easily escapable as shown below:



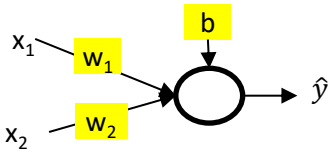
Plateau

- Plateau is a region where the gradient is close to zero.
- Slows down gradient descent significantly.



Gradient descent spends a long time searching its way here.

Parameter vs Hyperparameter

Parameter	Hyperparameter
• Defines the model, e.g., **w**, *b* in Logistic Regression. • The target of the training process • Their values are determined the from training process. 	• Training or network settings. • Not the target of training. • Handpicked by the user before training • Example: learning rate, no. of layers, no. of neurons in each layer

Thank you

for your attention